Hardening Methodology

Operation fsoc: Case Study

Gurmann Basran

Revised 2026-06-15 (rev. 3).

1 Why I wrote one

When I started this project, my first few sessions were "read a blog post, copy a config, move on." That produced a system that looked hardened. My notes said I had done a lot of things. In practice, three weeks later, I could not tell you which change addressed which risk, or whether a given rule was still serving its original purpose or had become vestigial. The methodology in this document exists because I got tired of that.

The core idea is borrowed from formal audit practice and scaled down to a solo operator: every finding gets a stable ID, every mitigation names the ID it closes, and no "fix it later" entry survives without a phase assignment. It sounds bureaucratic when you first read it and it sounds obvious after you have done it. The difference is measured in how long it takes to answer "why does this rule exist?" three months from now.

This is the audit-and-verification half of a broader systems-engineering discipline the project runs on. There is a charter, a requirements specification in which each requirement carries a stable identifier, and a traceability matrix that already links each requirement to the design that satisfies it and the test that proves it. The finding IDs in this document plug into that same chain, so a fix is never just a change; it is a change tied back to the requirement it serves and the verification that confirms it. The point of the whole apparatus is that traceability, not the paperwork for its own sake.

2 The five-step loop

2.1 Step 1: Red-team the existing configuration

I run two passes: outside-in and inside-out. Outside-in asks what an attacker on the WAN sees. Inside-out asks what an attacker on a compromised container, or a compromised home-zone device, sees. The inside-out pass is the one that catches the interesting stuff, because it mirrors what actually happens during a real breach: the first point of entry is rarely the most interesting compromise.

The red-team pass enumerates every exposed service, every allow-all rule, every credential that is not handled the way it should be, every default binding, and every cross-zone path. The output is a flat list of observations. It is deliberately uncurated; I do not try to prioritize while I am discovering.

2.2 Step 2: Log every finding with a stable ID

Each observation becomes a finding of the form `{ID, description, severity}`. The ID is stable for the lifetime of the project; it appears in commit messages, phase document headers, and the audit trail. Severity is one of Critical, High, Medium, or Low, using the definitions in the Threat Model document. In the later fleet-wide audit I added an action tag to each finding as well, because by then the question I needed answered was not only "how bad is this" but "what do I actually do with it."

I deliberately do not fix anything at this step. The goal is to get the whole list written down before I start making judgment calls, because the prioritization step makes more sense with the whole picture in view.

2.3 Step 3: Map every finding to the phase that closes it

Every finding gets an assignment to a specific phase in the roadmap. If a finding has no phase, I have two choices: create a phase, or mark it as residual risk in the threat model and explain why it is acceptable. What I do not do is leave it floating. "I will get to this later" is how security work silently rots.

This step is also where I catch findings that belong in a completely different phase than I would have guessed. What looks like a DNS-layer problem might really be a supply-chain problem; what looks like a firewall-rule problem might really be a naming problem, where I cannot tell which services live in which zone. The mapping forces me to be precise about what I am actually fixing.

2.4 Step 4: Leave the paper trail in commits

Every mitigation commit names the finding ID it closes. The mechanism is a commit message like `+close H3: tighten inter-zone policy on management path`. Months from now, when I am reading the log and wondering why a particular rule exists, the commit message tells me the finding it addressed, and the finding itself tells me what risk it was mitigating.

This is the step that unlocks the real value of the methodology. Without it, the finding list is a to-do list; with it, the finding list becomes a history.

2.5 Step 5: Gate later phases on earlier phases landing

If a later phase depends on an earlier phase's firewall rules being in place, the later phase does not begin until the earlier phase is marked complete and its audit findings are closed. I do not speculatively work on dependent layers; the cost of rework is high and the intervening soak time catches the things that were wrong in the earlier phase but did not surface immediately.

This is also where I learned that 24 hours of soak between a hardening phase and the next one is non-negotiable. I have more than once thought a change worked, only to discover 12 hours later that something did not like it. A phase gate is a forcing function for that soak time.

3 The finding table in practice

I have run this loop twice at very different scales. The first time was an early hardening pass that produced several dozen findings spread across Critical, High, Medium, Low, and a red-team-additional category from a second, more adversarial review. The *structure* of that table, with the contents stripped because they map to specific services, looks like this:

ID	Severity	Category	Phase that closes it
C1	Critical	Admin plane	Early sub-phase of hardening pass
C2	Critical	Admin plane	Early sub-phase of hardening pass
C3	Critical	Credential management	Early sub-phase of hardening pass
…	…	…	…
H1	High	Zone isolation	Mid sub-phase of hardening pass
…	…	…	…
RT-A	High (red-team)	DNS security	Mid sub-phase of hardening pass
…	…	…	…

The second time, I ran the loop across the whole fleet as one structured milestone, and that is where the method earned its keep. I split the review into separate domain passes, one per area of the system, each pass observe-only and each finding carrying command-output evidence rather than a hunch. That audit produced close to a hundred findings across seven domains: authentication, container security, firewall and network policy, the integrity of the scripts and their scheduling, the monitoring and detection chain, the state of the documentation against reality, and dead or dangling references left behind by earlier work. By then a severity letter was not enough to drive the work, so every finding also got an action tag: fix it now, improve it, remove it, or defer it as accepted residual risk. The split came out roughly a third fix, a little under half improve, and the remainder split between remove and defer.

The fix-tagged findings are the ones that became the next milestone. There were twenty-nine of them, and the remediation milestone closed every one. The discipline I most want to show is what "closed" means here: each fix-required finding is traced from its ID through the phase that owned it, the plan that executed it, and the verification test that proved it, and the milestone did not get marked done until that trace read twenty-nine of twenty-nine satisfied. The improve-tagged findings became a separate hardening milestone layered on top, and the defer-tagged ones went into the threat model's residual-risk column with a reason attached, not into a void.

The point of showing the structure rather than the contents is the same at both scales: the structure is what transfers to another project. The specific findings are project-specific; the methodology is not.

4 The dimension the audit did not have

The fleet-wide audit covered seven domains, and I was quietly proud of how complete it felt. Then the system failed in a way the audit had no language for. A background process went runaway and pinned a processor core for the better part of a week before I caught it; on a separate occasion a routine maintenance job staged far more data into a memory-backed scratch area than the host had memory for, and drove that host into the kernel's out-of-memory killer while it was otherwise sitting idle. Neither of those was a security misconfiguration. Both of them took a service down. And every domain of the audit would have passed clean the entire time.

The gap was that every question the audit asked was a security question: can an attacker reach this, is this surface exposed, is this credential scoped to what uses it. None of them was an operational question: does this host have headroom, is something consuming a resource without bound, will this stay up under its own legitimate load. A static configuration audit, run at a point in time, cannot answer those, because they are temporal. The failure is not visible in any single snapshot; it lives in a trend across hours. A snapshot taken an hour before the out-of-memory kill looks perfectly healthy.

So I added an eighth dimension to the methodology, operational health and capacity, and I deliberately gave it a different shape from the other seven. Its findings are tagged by the failure mode they expose, runaway process, memory or swap pressure, unbounded growth, filesystem fill, rather than by an attacker-facing severity. And the dimension is not satisfied by a one-time pass the way a configuration domain is; it is satisfied by a watchdog that keeps asking the question on a tight interval, from a host that does not depend on the one being measured. The watchdog itself is described in the scripts document, and the integration lesson that produced it is in the lessons document. The rest of the methodology around it did not change at all: each finding still gets a stable ID, an action tag, a phase assignment, and a trace to the change that closed it. What changed is that I stopped quietly assuming that a system which is secure is the same thing as a system which is healthy. The same realization is why a later milestone went back and proved, rather than assumed, that the backups restore; assuming a property holds because the configuration looks right is exactly the trap this dimension exists to break.

5 Phase ordering: safe changes first

The single most valuable lesson from running the first pass of this methodology was that change ordering matters as much as the changes themselves. An earlier draft of my hardening plan nearly cost me a multi-hour internet outage because I sequenced a dangerous reconfiguration of the perimeter before the safety net was in place.

The refined ordering I use now has four tiers:

1. **Pre-flight.** Backups of every VM, a dead-man SSH session open to an emergency IP on a separate physical NIC, physical console access verified. None of these change the running system; they are insurance.

2. **Safety-net changes.** Things that cannot break connectivity. Credential hardening, secret rota-

tion, internal encryption-at-rest. If one of these goes wrong, the service affected is degraded; nothing catastrophic happens.

3. **Reversible changes.** Zone-policy tightening, service binding changes, SSH configuration. Each has an explicit rollback plan. These run in a single session with a second SSH window open so I can roll back if the primary session dies.

4. **Dangerous changes.** Anything that reconfigures the admin plane itself: firewall GUI binding, SSH access policy, VPN policy. These require a human-verified post-flight checklist and at least one live second SSH session for rollback.

Cosmetic cleanup (stale host entries, config comments, service banners) happens last, after the hardening phase has soaked for long enough to know everything still works.

Every phase declares its own rollback plan before execution. If there is no rollback plan, the phase does not run. This sounds strict until you have nearly lost internet for an afternoon because you skipped writing one.

6 Phase gating in practice

This is the shape of a phase-gate checklist as it appears in the live planning document, redacted:

```
Pre-flight checklist for the next hardening phase.
All items must be checked before the phase begins.

[ ] Second persistent SSH session open via emergency IP (dead-man switch)
[ ] Physical console access verified; keyboard and monitor ready
[ ] Snapshot of the firewall VM and the services VM completed
[ ] Firewall config backup downloaded and checksum-pinned
[ ] All prior-phase findings marked CLOSED
[ ] 24-hour soak test since the prior phase with no untriaged alerts
```

If any box is unchecked, the phase does not begin. Not tomorrow; not after I grab coffee; the phase is blocked until the box is green. The handful of times I have been tempted to skip a gate have all been cases where a gate was most likely to save me.

7 Why this matters

The difference between a system that looks hardened and a system that actually is hardened is traceability. When something breaks three months from now, the right question is not “what did we do?” It is “which finding did this change close, and what was the rollback plan?” The answer has to exist in writing, in the commit log, still findable. This methodology is how you make that answer exist.

The second reason to do this is that hardening work, like any skill, improves with practice. The only way to improve is to make the process legible enough to critique afterward. A methodology I

can read back is a methodology I can improve; a set of ad-hoc changes I made on a Tuesday at 11 PM is not. Running the same loop a second time, at fleet scale, and watching it close twenty-nine of twenty-nine fix-required findings with a verifiable trace, is the closest thing to proof that the legibility was worth the bureaucracy.